sync.py × Terminal

```
1
2  import os
3  import openai
4  import requests
5  import secret
6  import time
7
8  openai.api_key='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'
9  # WRITE YOUR CODE HERE
```

100% (9:1)

async.py ×

```
1
2  import os
3  import openai
4  import aiohttp
5  import asyncio
6  import secret
7  import time
8
9  openai.api_key = secret.api_key
10 # WRITE YOUR CODE HERE
```

100% (2:1)

Guide ×

Collapse

Async Example

```
        print(f"Unexpected error: {e}")
```

main function: This function creates an `asyncio` task for each prompt, adds them to a list, and then runs them concurrently using `asyncio.gather()`. This means that it will start all the tasks at the same time, and then wait for all of them to finish. This is where the asynchronous magic happens - rather than processing each prompt one by one, it starts processing all of them at once, which can significantly speed up the total execution time. It also measures the execution time by recording the time before and after the tasks are run, and then subtracts the start time from the end time.

```
async def main():
    start_time = time.time()  # Start measuring execution time

    tasks = []
    for i, prompt in enumerate(prompts):
        tasks.append(fetch_image(prompt, i))

    await asyncio.gather(*tasks)

    end_time = time.time()  # End measuring execution time
    execution_time = end_time - start_time
    print(f"Execution time: {execution_time} seconds")
```

Your whole code outline in `async.py` should look like the following code. Additionally, to run the main function we have added `asyncio.run(main())` at the end of our code.

```
import os
import openai
import aiohttp
import asyncio
import secret
import time

openai.api_key = secret.api_key

# Set the prompts
prompts = ["robot dog in a lab", "robot dog exploring the city", "robot dog watchin

async def fetch_and_save_image(session, url, path):
    try:
        async with session.get(url) as resp:
            img_data = await resp.read()
            with open(path, 'wb') as handler:
                handler.write(img_data)
    except Exception as e:
        print(f"Unexpected error: {e}")

async def fetch_image(prompt, i):
    try:
        response = openai.Image.create(
            prompt=prompt,
            n=1,
            size="256x256"
        )

        # Get the image URL from the response
        image_url = response['data'][0]['url']

        # Download and save the image
        async with aiohttp.ClientSession() as session:
            await fetch_and_save_image(session, image_url, f"robot_dog_journey_{i+1
    except openai.api_errors.APIError as e:
        # This will catch any error returned by the OpenAI API
        print(f"API error: {e}")

    except Exception as e:
        # This is a catch-all for any other exceptions
        print(f"Unexpected error: {e}")

async def main():
    start_time = time.time()  # Start measuring execution time

    tasks = []
    for i, prompt in enumerate(prompts):
        tasks.append(fetch_image(prompt, i))

    await asyncio.gather(*tasks)

    end_time = time.time()  # End measuring execution time
    execution_time = end_time - start_time
    print(f"Execution time: {execution_time} seconds")

# Run the main function
asyncio.run(main())
```

Use the Try It buttons below to check and compare the sync vs async time.

TRY IT SYNC

TRY IT ASYNC

Please note that the `openai.Image.create` function is a blocking operation and doesn't support async. The code above assumes that this function is asynchronous, which is not correct. To truly parallelize this, you'd need an async-compatible OpenAI library or use something like Python's `concurrent.futures` to run the blocking parts in a separate thread.

What is the primary advantage of using asynchronous programming when dealing with multiple API calls?

- ○ It reduces the complexity of the code.
- ◉ It allows for concurrent execution, which can significantly speed up the total execution time.
- ○ It prevents any errors from occurring during API calls.
- ○ It automatically handles any rate-limiting issues with the API.

It allows for concurrent execution, which can significantly speed up the total execution time. Asynchronous programming allows an application to start an operation, such as an API call, and then move on to another task without waiting for the first operation to complete. When the operation completes, the application is notified and can handle the result.

Check It!

Next